

```
>>> config1.sections()
[]
>>> config1.read("c:\\example.cfg")
['c:\\example.cfg']
>>> config1.sections()
['General', 'customerx', 'customery']
>>> config1.options('General')
['customer']
>>> config1.options('customerx')
['brandingtitle', 'customercontact']
>>> config1.has_option('customerx', 'brandingtitle')
True
>>> config1.has_option('customerx', 'phonenumber')
False
>>> config1.get('customerx', 'brandingtitle')
'CustomerX Title'
```



Note 1: Strings can be represented using single quotes ('General') or double quotes ("c:\\example.cfg").

Note 2: ConfigParser treats all values as strings. This is an important distinction if you want to treat the values as different data types, depending upon usage. You will need to convert it using built-in converters like *int()*, *float()*, etc.

Note 3: ConfigParser treats all values as strings, which also implies that when you want to set values into a config file, you will have to convert them using *str()* so that they can be written.

A procedure to check a config file

Now that we have the basic tools, let us put together a function to validate a given config file. To do so, we need to know the correct version of the config file, which we shall call the golden file. We want to be able to use a ConfigParser object to read from the golden file and then use that information to check the second config file. To keep the interface simple, we could simply take both files as arguments to the proposed function.

The proposed interface will look like what's shown below:

```
def ValidateConfigFile(goldenfilepath = 'none',
filetobevalidated = 'none'):
```

As an alternate, we could incorporate the golden file inside our program. This would avoid the need to distribute the golden file. But in such a design, whenever the config file changes to include newer parameters, it will necessitate changing our validation program to incorporate these changes. This, apparently, is not a good design.

Coming back to the API, here are a few pointers:

- Defining a function is done using the *def* keyword.
- The type of the parameters need not be defined and is

detected to be a string, in this case.

- We can assign default values for the parameters in case the caller does not pass the same. In this case, we use *'none'* as the default value to do some input parameter validation.
- It is also Python syntax to indicate the beginning of a block of code, in this case, a function, by ending the declaration with a colon :
- The same applies for other constructs like *if*, *for*, etc.



Note: Python uses indentation to indicate that a set of lines are to be treated as part of a block. There is no end statement for the block of code. In the case of our function, the whole function needs to be indented by one level to indicate that the set of lines belong to the function.

Now, let us use a couple of ConfigParser objects and get our processing done. We have an erroneous config file created in the following manner, where the red lines are invalid:

```
#error.cfg
[General]
Custom = CustomerX
[customerx]
BrandingTitle=CustomerX Title
CustomContact=CustomerX contact
[customery]
BrandingTitle=CustomerY Title
tomerContact=CustomerY contact
```

Now, let's create a ConfigParser object for the golden file and another for the file to be checked:

```
#parse golden file
goldenconfig = ConfigParser.ConfigParser()
goldenconfig.read(goldenfilepath)

#parse file to be validated
filetovalidate = ConfigParser.ConfigParser()
filetovalidate.read(filetobevalidated)
```

Let us run through those lines in IDLE GUI:

```
>>> goldenconfig = ConfigParser.ConfigParser()
>>> goldenconfig.read(goldenfilepath)
['c:\\example.cfg']
>>> filetovalidate = ConfigParser.ConfigParser()
>>> filetovalidate.read(filetobevalidated)
['c:\\error.cfg']
```

For each option or key in a section, we again confirm that the same exists in the *goldenconfig*. We can create a simple list of strings with which we can store invalid lines when they occur (called *incorrectlines*) if the above check fails. A list has